4. BÚSQUEDA ENTRE ADVERSARIOS
(PARTE 1)

IA 3.2 - Programación III

1° C - 2026

Dr. Mauro Lucci

Agente

- Un **agente** es cualquier cosa capaz de percibir su **entorno** con la ayuda de **sensores** y actuar en ese medio utilizando **actuadores**.
- 📖 **Ejemplos.**
  - Agente humano. Sensores: ojos, oídos, etc. Actuadores: manos, piernas, boca, etc.
  - Agente robot. Sensores: cámara, láser, gps, etc. Actuadores: motores.
  - Agente de software. Sensores: teclado, mouse, lector de archivos, placa de red. Actuadores: mostrar por pantalla, escribir archivos, enviar paquetes de red.
- En las unidades pasadas estudiamos agentes de software que resuelven problemas de búsqueda.

Entornos multiagentes

- En un entorno **multiagente**, el comportamiento de un agente depende del comportamiento de los demás.
- Es **competitivo** si los objetivos de los agentes están en conflicto.

  📖 **Ejemplo**. Ganar una partida de truco, implica que el otro jugador la pierde, y viceversa.
- De lo contrario, se dice **cooperativo**.

  📖 **Ejemplo**. En el contexto de un taxi autónomo, evitar choques es un objetivo común entre todos los agentes del tránsito.
- La **teoría de juegos** es una rama de la economía y estudia este tipo de entornos.

🎲 Juegos

Un problema de **búsqueda entre adversarios**, o **juego**, involucra un entorno multiagente y competitivo. Formalmente, se define por:

1. **S_0**. **Estado inicial**.
2. **JUGADOR(S)**. Función que determina qué jugador tiene el turno en ese estado.
3. **ACCIONES(S)**. Función que devuelve el conjunto de **acciones** legales en ese estado.
4. **RESULTADO(S,A)**. Es el **modelo de transiciones**, que define el estado resultado de aplicar esa acción en ese estado.

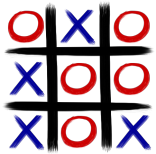
🎲 Juegos

---

5. **TEST-TERMINAL(S).** Función que determina si el juego está terminado en ese estado. Los estados en donde el juego ha finalizado se llaman **estados terminales**.

6. **UTILIDAD(S,P).** **Función de utilidad** que asigna un valor numérico (recompensa) a cada jugador P en un estado terminal S.

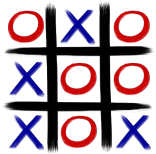
   📖 **Ejemplo.** En ajedrez la utilidad puede ser 1, 0 o ½ según si el jugador gana, pierde o empata.

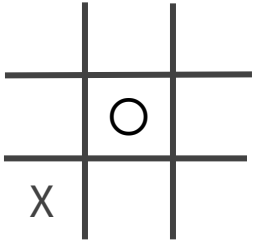
📖 Ejemplo – TA-TE-TI 

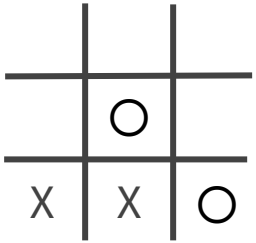
En esta formulación, un **estado** es una matriz M de 3x3 donde, para cada $i,j \in \{1,2,3\}$, la casilla $M[i][j] \in \{-,X,O\}$.

El número total de estados es 3^9 = **19683**. Sin contar estados *simétricos* son **765**.

📖 Ejemplo – TA-TE-TI





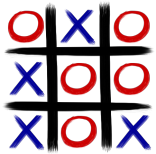


Representación.

–	–	–
X	O	–
X	–	–

–	–	–
–	O	–
X	X	O

📖 Ejemplo – TA-TE-TI



X	O	
X		

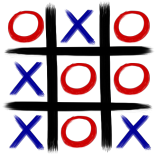
	O	
X	X	O

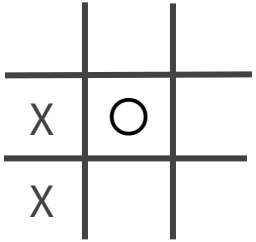
Representación.

–	–	–
X	O	–
X	–	–

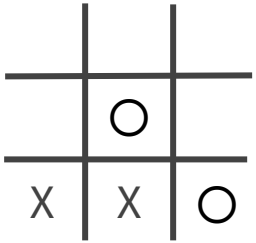
–	–	–
–	O	–
X	X	O

📖 Ejemplo – TA-TE-TI





Turno de **MIN**



Turno de **MAX**

En esta representación, es posible **deducir el turno** a partir del estado actual.

MAX: jugador inicial.
MIN: jugador no inicial.

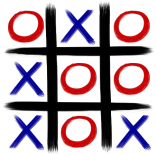
- Si hay misma cantidad de Xs que de Os ⇒ **MAX**
- Si hay mayor cantidad de Xs que de Os ⇒ **MIN**

Representación.

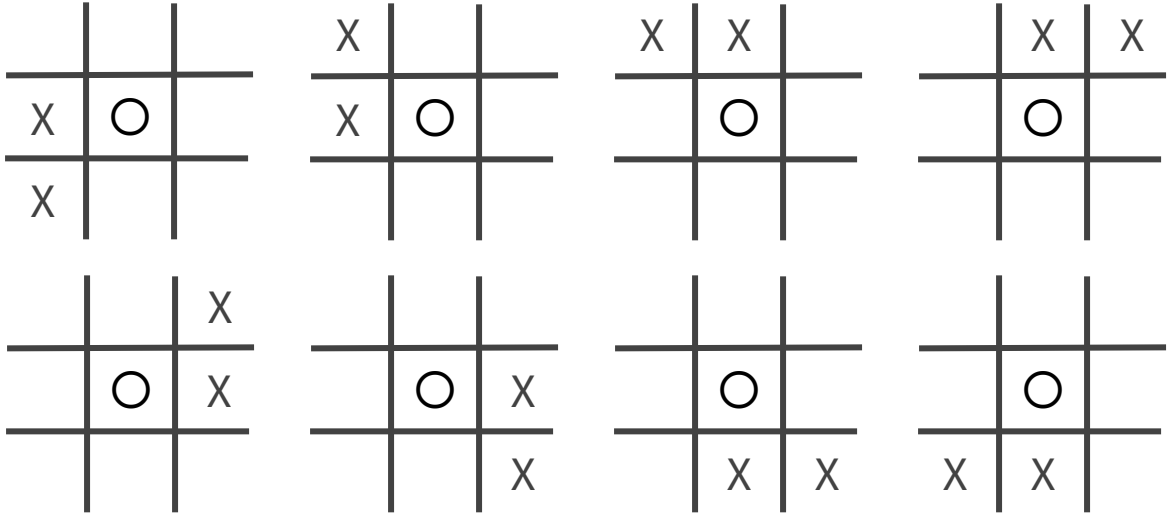
–	–	–
X	O	–
X	–	–

–	–	–
–	O	–
X	X	O

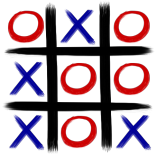
📖 Ejemplo – TA-TE-TI



Los siguientes tableros pueden considerarse como un mismo estado, dado que son **simétricos** (por rotaciones y reflexiones).

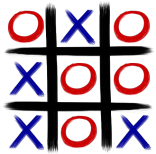


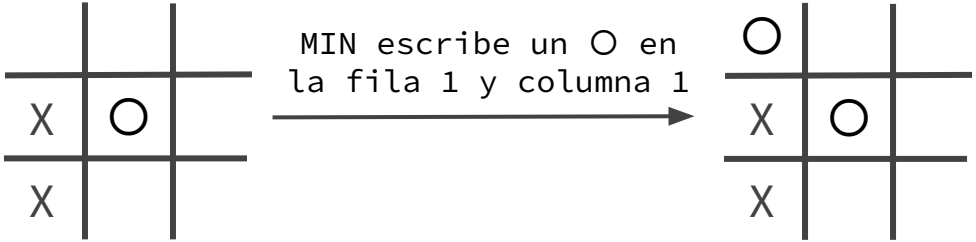
Por el momento, nuestra formulación no distinguirá estados simétricos

📖 Ejemplo – TA-TE-TI 

Una **acción** es un par $(i,j) \in \{1,2,3\}^2$ que indica que se escribe una X o un O (según el jugador que corresponda) en la casilla $M[i][j]$.

📖 Ejemplo – TA-TE-TI



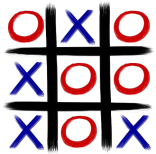


Representación.

–	–	–
X	O	–
X	–	–

(1,1) →

O	–	–
X	O	–
X	–	–

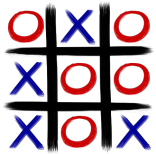
📖 Ejemplo – TA-TE-TI 

---

1. **S_0**: Matriz M tal que M[i][j] = -, para todos i,j ∈ {1,2,3}.
2. **JUGADOR.** Dado un estado M,

$$\text{JUGADOR}(M) = \begin{cases} \text{MAX} & \text{si M tiene misma cantidad de Xs que Os.} \\ \text{MIN} & \text{en caso contrario.} \end{cases}$$

En particular, JUGADOR(S_0) = MAX.

📖 Ejemplo – TA-TE-TI 

---

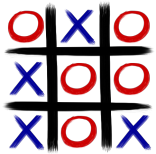
3. **ACCIONES.** Dado un estado M,

   ACCIONES(M) = $\{(i,j) \in \{1,2,3\}^2 : M[i][j] = -\}$

4. **RESULTADO.** Dado un estado M y una acción $(i,j) \in$ ACCIONES(M),

   RESULTADO(M,(i,j)) = M' donde, para cada $i',j' \in \{1,2,3\}$,

$$M'[i'][j'] = \begin{cases} X & \text{si } i' = i,\ j' = j \text{ y JUGADOR(M) = MAX.} \\ O & \text{si } i' = i,\ j' = j \text{ y JUGADOR(M) = MIN.} \\ M[i][j] & \text{en caso contrario.} \end{cases}$$

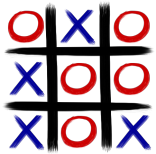
📖 Ejemplo – TA-TE-TI 

---

5. **TEST-TERMINAL.** Dado un estado M,

   TEST-TERMINAL(M) = True si y sólo si M cumple al menos una de las siguientes condiciones.

   - **No hay casillas libres:**
     M[i][j] ≠ -, para todo i,j ∈ {1,2,3}.
   - Tres iguales en **fila:**
     M[i][1] = M[i][2] = M[i][3], para algún i ∈ {1,2,3}.
   - Tres iguales en **columna:**
     M[1][j] = M[2][j] = M[3][j], para algún j ∈ {1,2,3}.
   - Tres iguales en **diagonal:**
     M[1][1] = M[2][2] = M[3][3] o M[1][3] = M[2][2] = M[3][1].

📖 Ejemplo – TA-TE-TI 

---

6. **UTILIDAD.** Dado un estado terminal M y un jugador P,

$$\text{UTILIDAD}(M,P) = \begin{cases} 1 & \text{si P gana} \\ ½ & \text{si P empata} \\ 0 & \text{si P pierde} \end{cases}$$

MAX gana y MIN pierde si hay 3 Xs en línea, MIN gana y MAX pierde si hay 3 Os en línea y empatan en caso contrario.

Tipos de juegos

Existe una amplia variedad de juegos. **Nos limitaremos a juegos con:**

- **Dos jugadores** que juegan por **turnos**. Los jugadores se llaman **MAX** y **MIN**, siendo MAX quién juega primero. Luego juega MIN, luego MAX de vuelta, y así sucesivamente.

- **Suma cero**. La suma de las utilidades de los jugadores es la misma para todo estado terminal.

  📖 **Ejemplo**. El ajedrez es un juego de suma cero porque las utilidades en los estados terminales suman 1 (gana-pierde: 1+0, pierde-gana: 0+1, empatan: ½ + ½).

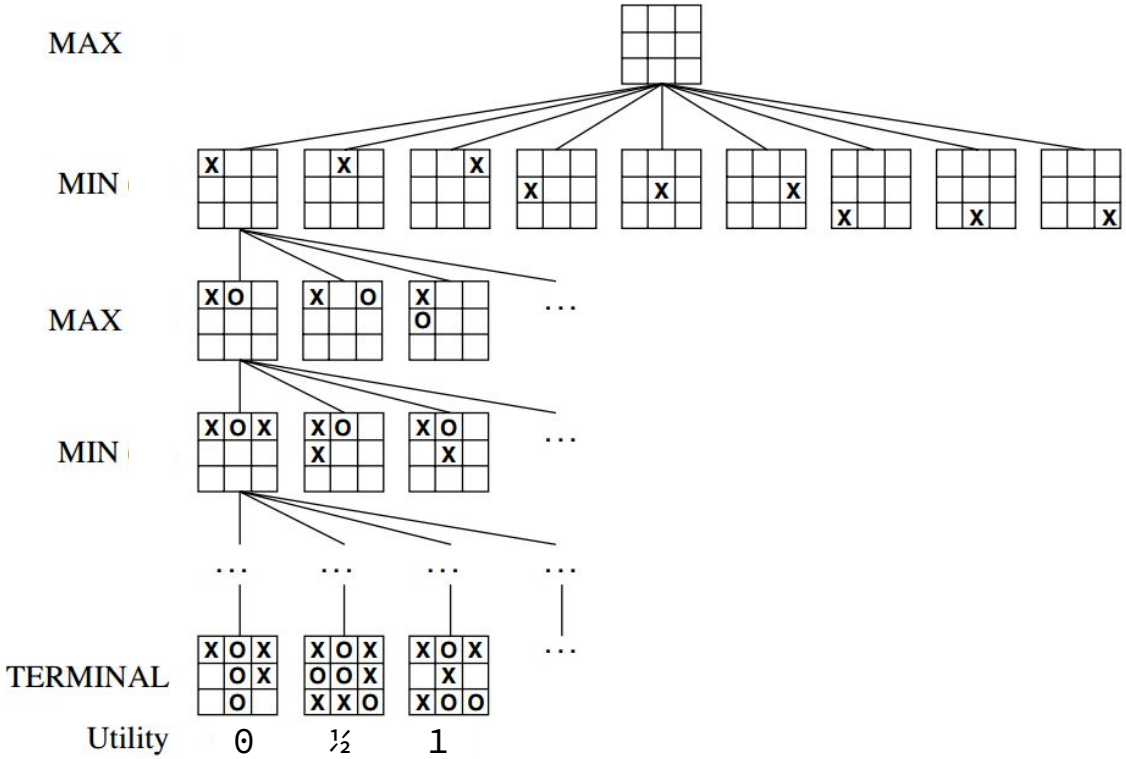
- **Información perfecta**. Al tomar una decisión, cada jugador está perfectamente informado de todos los eventos ocurridos previamente, incluida la inicialización.

  📖 **Ejemplo**. El ta-te-ti tiene información perfecta. Por el contrario, el poker tiene información **imperfecta**, ya que no se ven las cartas de los otros jugadores.

Árbol de juego

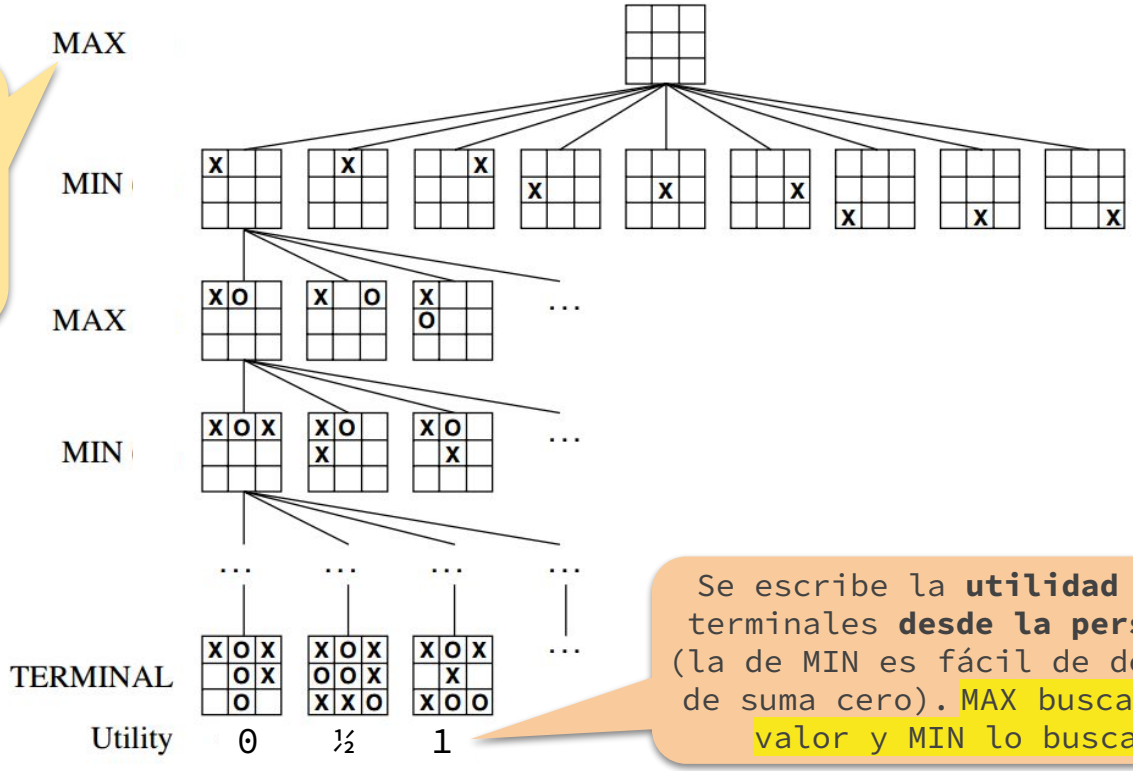
El estado inicial, las acciones y el modelo de transiciones definen un **árbol de juego**, donde los nodos representan estados y las ramas representan acciones, similar a un árbol de búsqueda.

📖 Ejemplo – TA-TE-TI



📖 Ejemplo – TA-TE-TI

Los niveles indican el turno de juego. En los niveles pares es el turno de MAX y en los impares el de MIN.



Se escribe la **utilidad** de los estados terminales **desde la perspectiva de MAX** (la de MIN es fácil de deducir en juegos de suma cero). MAX busca **maximizar** este valor y MIN lo busca **minimizar.**

Solución de un juego

- Si el juego fuese un problema de búsqueda sin adversarios, una solución sería una secuencia de acciones desde el estado inicial a un estado objetivo, en este caso, un estado terminal donde ese jugador gane.

🚧 Sin embargo, cuando hay adversarios, **el otro jugador también aplica acciones** y hará lo posible para ganar él.

Estrategia óptima

- Resolver un juego implica hallar una **estrategia** que especifique **qué acción debe aplicar un jugador en cada uno de sus turnos hasta que el juego llegue a un estado terminal.**
- Una **estrategia óptima** produce un resultado al menos tan bueno como cualquier otra estrategia cuando se juega contra un oponente **infalible** (es decir, que juega **óptimamente**).

Valor minimax

- Dado un árbol de juego, el **valor minimax** de un nodo es la **mejor utilidad** que puede alcanzar MAX cuando el juego se encuentra en ese estado, asumiendo que ambos jugadores juegan **óptimamente** desde allí hasta el final del juego.
- **Los valor minimax permiten hallar una estrategia óptima.** Pero veamos primero cómo se definen estos valores.

Valor minimax

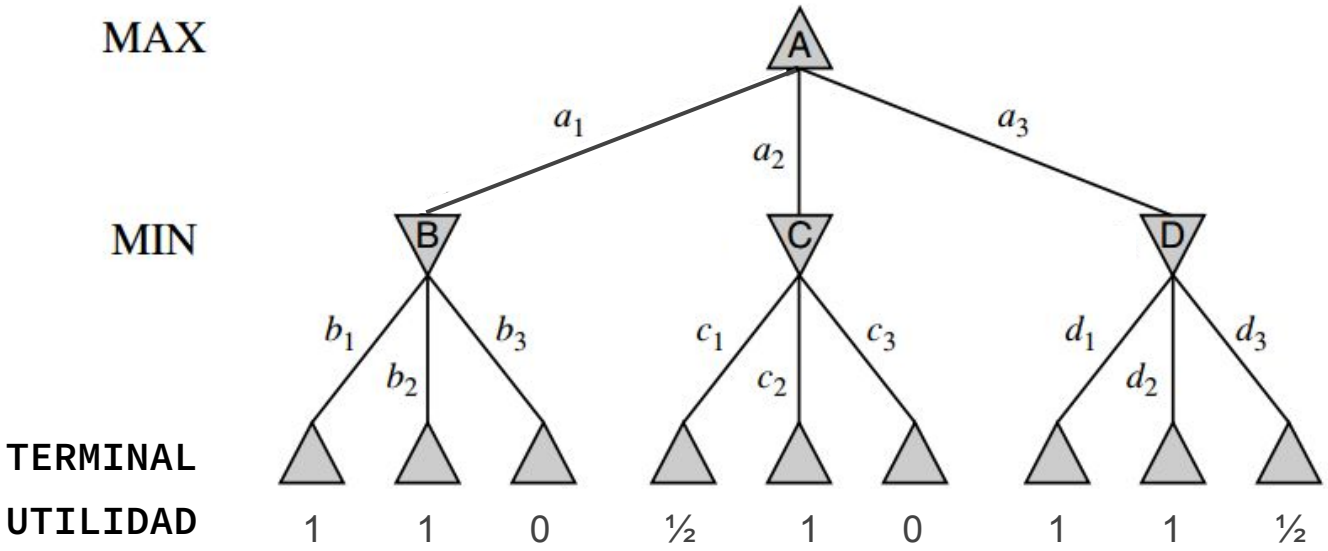
Las condiciones se deben evaluar en orden.

$$
\mathbf{MINIMAX}(S) = \begin{cases} \text{UTILIDAD}(S,\text{MAX}) & \text{SI TEST-TERMINAL}(S) \\ \max \{\mathbf{MINIMAX}(\text{RESULTADO}(S,A)) : A \in \text{ACCIONES}(S)\} & \text{SI JUGADOR}(S) = \text{MAX} \\ \min \{\mathbf{MINIMAX}(\text{RESULTADO}(S,A)) : A \in \text{ACCIONES}(S)\} & \text{SI JUGADOR}(S) = \text{MIN} \end{cases}
$$

- El valor minimax de un nodo con un estado terminal es su utilidad.
- MAX prefiere moverse a un nodo con mayor valor minimax.
- MIN prefiere moverse a un nodo con menor valor minimax.

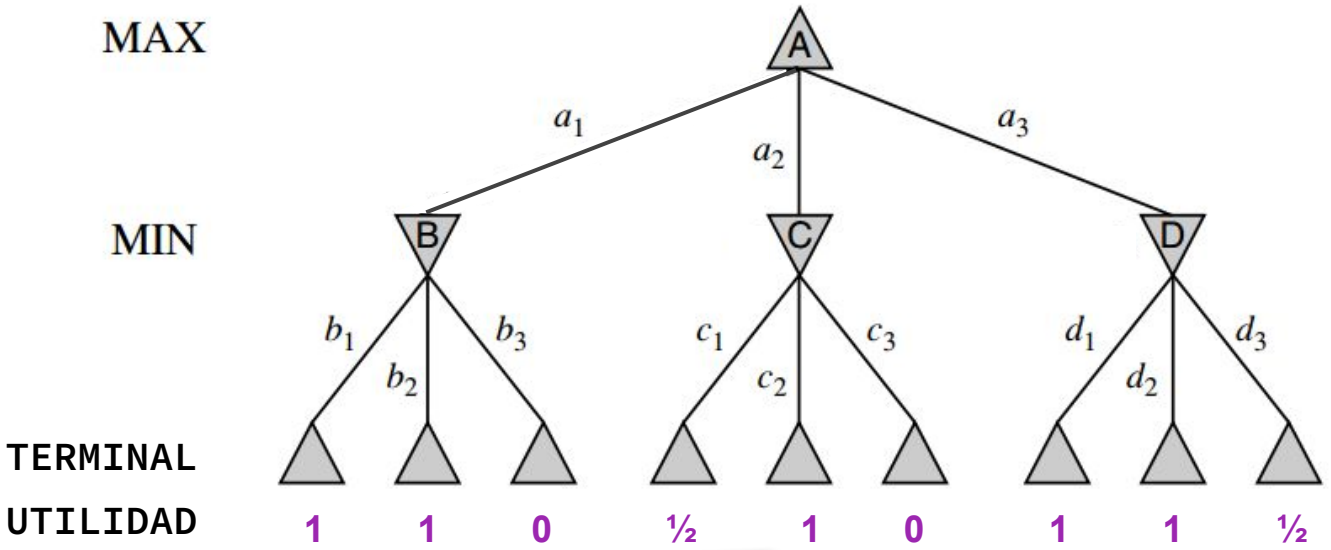
📖 Ejemplo

Vamos a computar el valor minimax de todos los nodos en el siguiente árbol de juego.



📖 Ejemplo

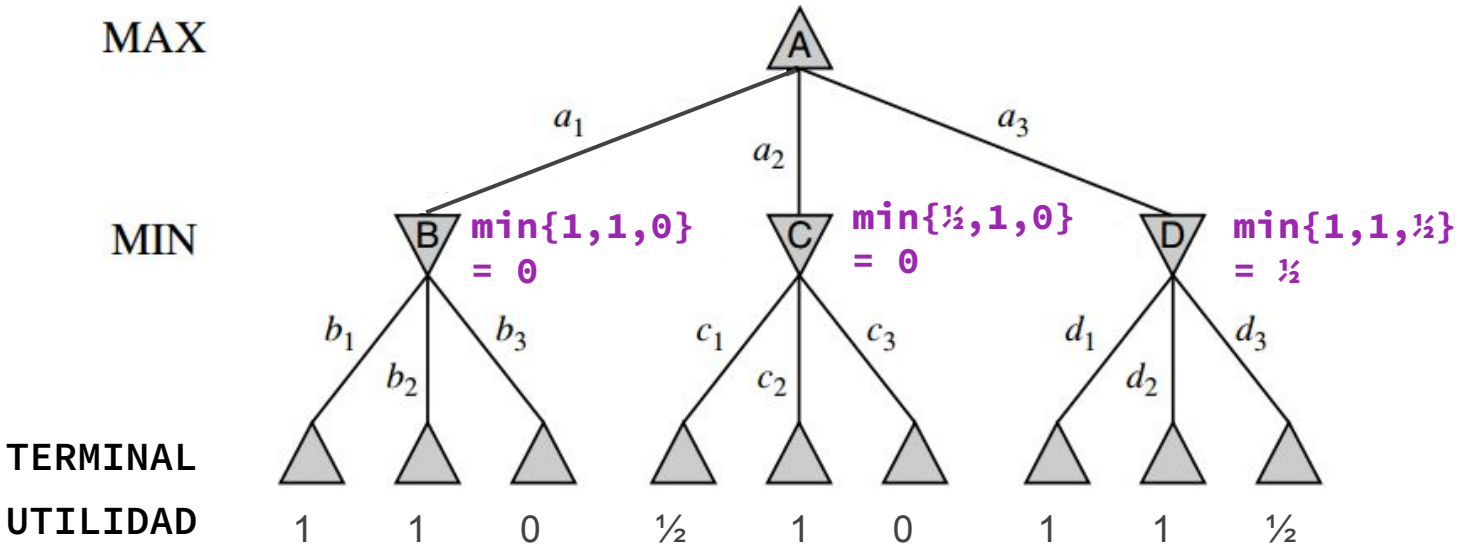
```
MINIMAX(S) =

    { UTILIDAD(S, MAX)                                  SI TEST-TERMINAL(S)
    { max {MINIMAX(RESULTADO(S,A)): A ∈ ACCIONES(S)}    SI JUGADOR(S) = MAX
    { min {MINIMAX(RESULTADO(S,A)): A ∈ ACCIONES(S)}    SI JUGADOR(S) = MIN
```



En las **hojas**, el valor minimax es directamente la utilidad. Luego, **escalamos** hasta la raíz.

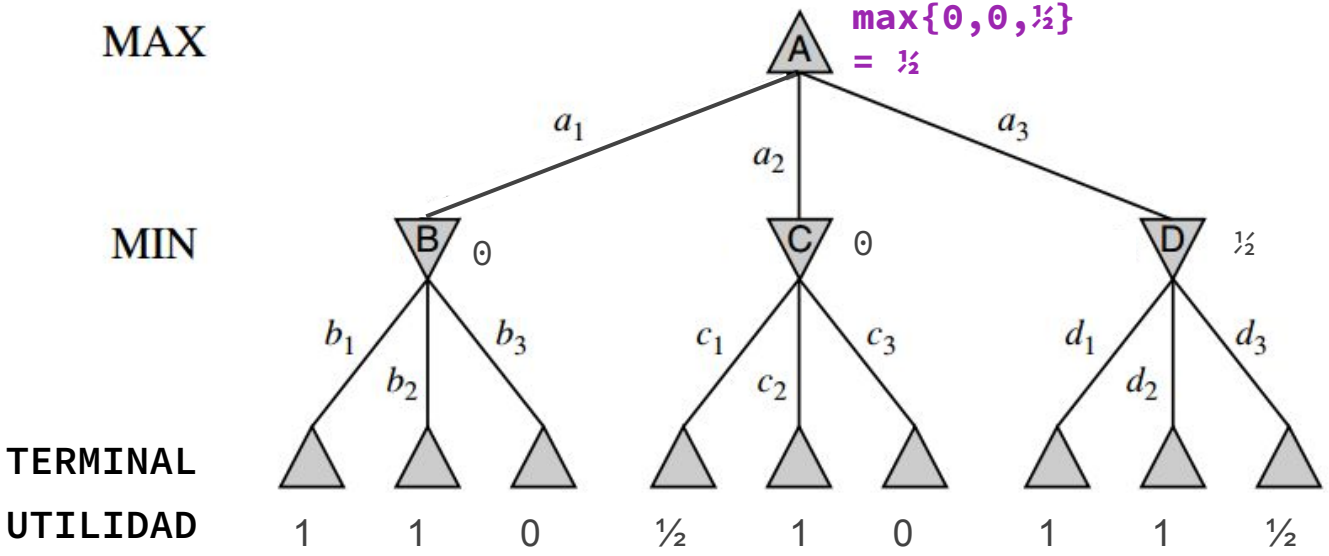
📖 Ejemplo

```
MINIMAX(S) =

    { UTILIDAD(S, MAX)                                  SI TEST-TERMINAL(S)
    { max {MINIMAX(RESULTADO(S,A)): A ∈ ACCIONES(S)}    SI JUGADOR(S) = MAX
    { min {MINIMAX(RESULTADO(S,A)): A ∈ ACCIONES(S)}    SI JUGADOR(S) = MIN
```

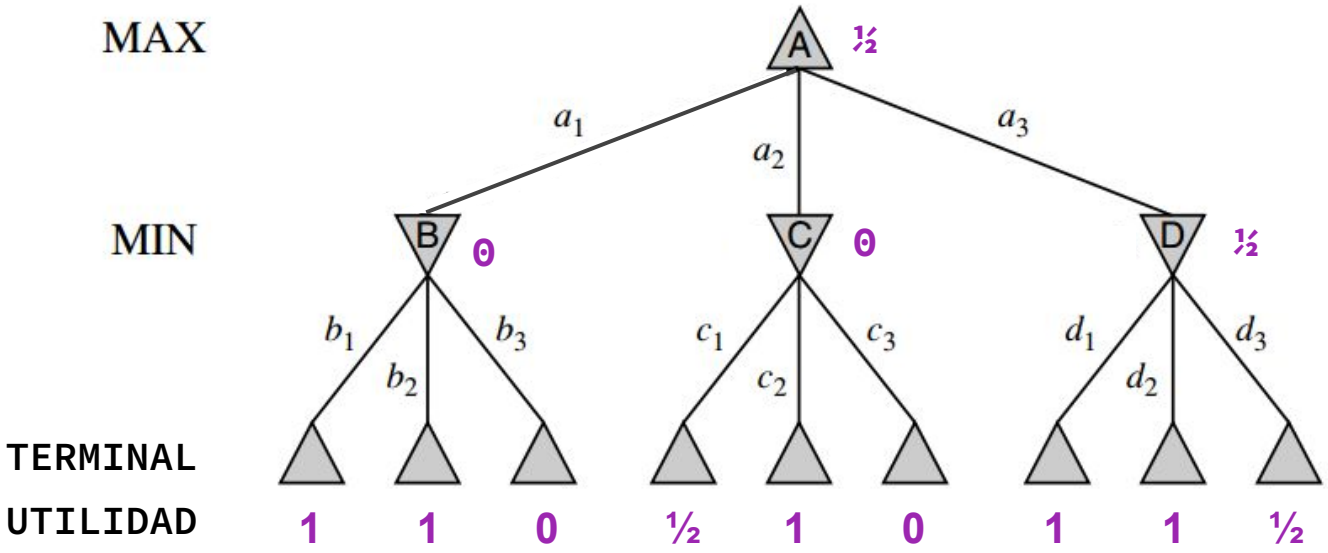


📖 Ejemplo

```
MINIMAX(S) =

    { UTILIDAD(S,MAX)                                   SI TEST-TERMINAL(S)
    { max {MINIMAX(RESULTADO(S,A)): A ∈ ACCIONES(S)}    SI JUGADOR(S) = MAX
    { min {MINIMAX(RESULTADO(S,A)): A ∈ ACCIONES(S)}    SI JUGADOR(S) = MIN
```



📖 Ejemplo

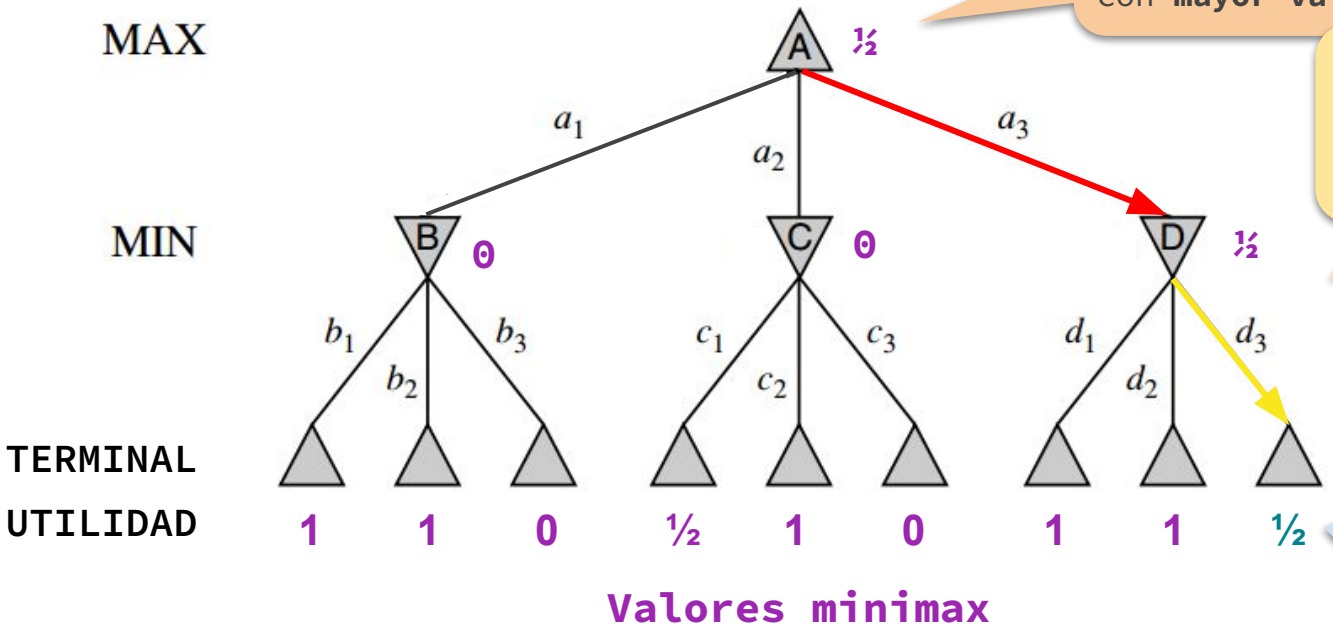


Valores minimax

📖 Ejemplo

La **estrategia minimax** dice que **MAX** elija la acción que lleva al hijo con **mayor valor minimax.**

Y dice que **MIN** elija la de **menor valor minimax.**



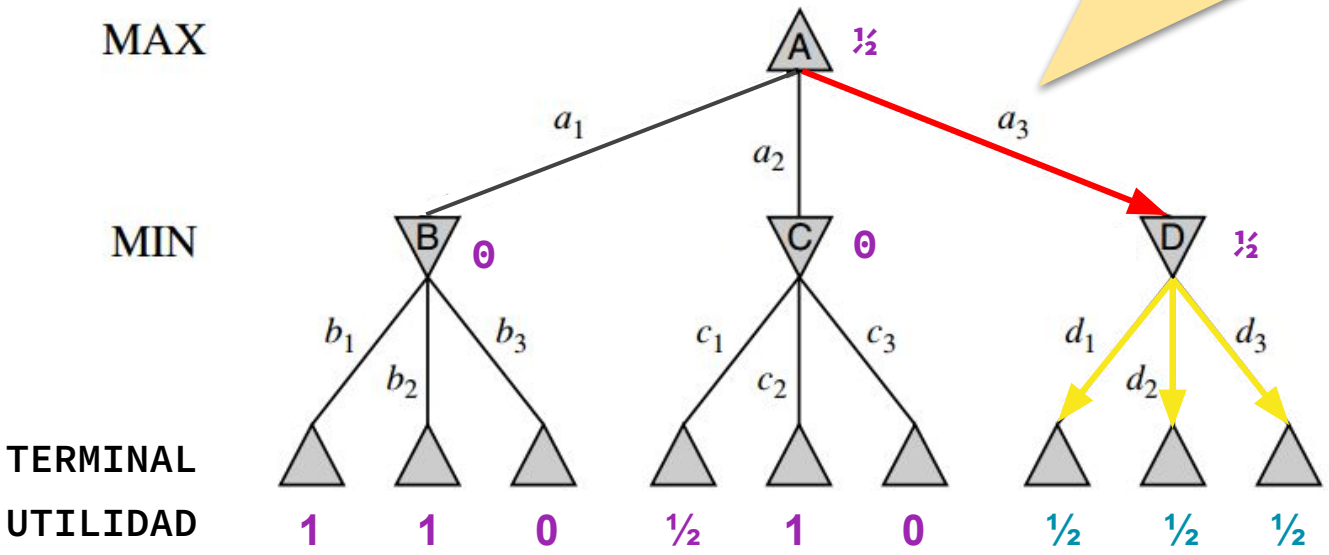
Para este juego, la estrategia minimax termina en **empate**.

Estrategia MINIMAX

- **Estrategia minimax:**
  - **MAX** elige la acción que lleva al hijo con **mayor valor minimax**.
  - **MIN** elige la acción que lleva al hijo con **menor valor minimax**.
- **La estrategia minimax es una estrategia óptima** (asumiendo que ambos jugadores juegan óptimamente).
- Es decir, otras estrategias podrían conducir a un peor resultado si el oponente juega óptimamente.
- Pero contra oponentes subóptimos, otras estrategias pueden ser incluso mejores que minimax. Veamos ejemplos…

📖 Ejemplo

Si MAX obedece la estrategia minimax y juega a_3, el juego termina inevitablemente en un **empate**, independientemente de la acción que realice MIN.

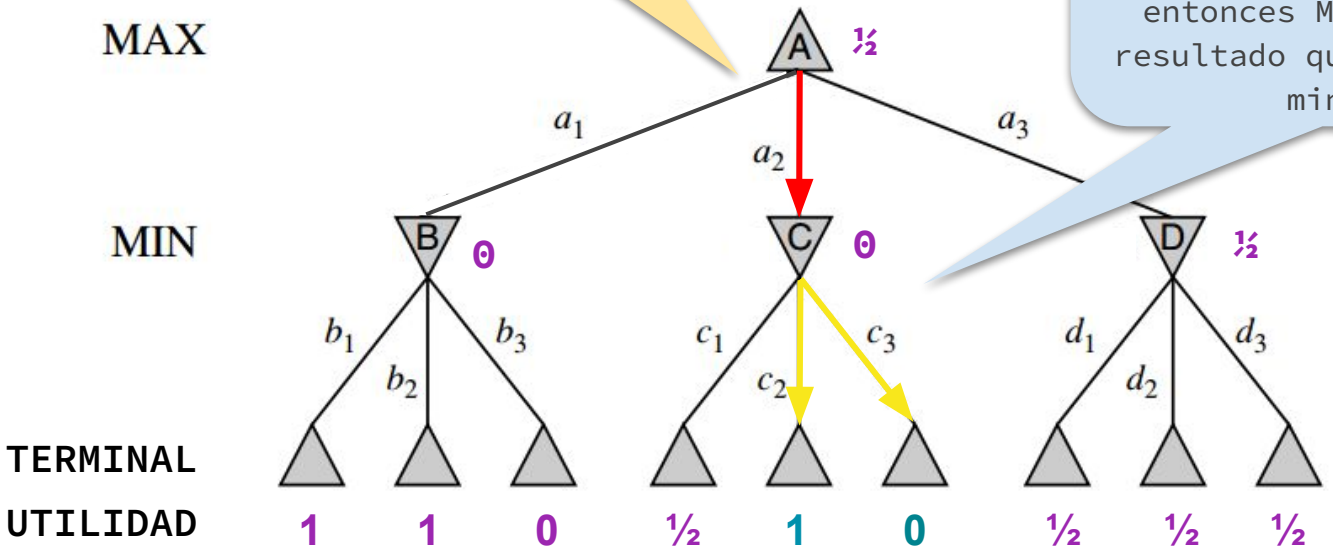


Valores minimax

📖 Ejemplo

Si MAX **no** obedece la estrategia minimax y juega a_2…

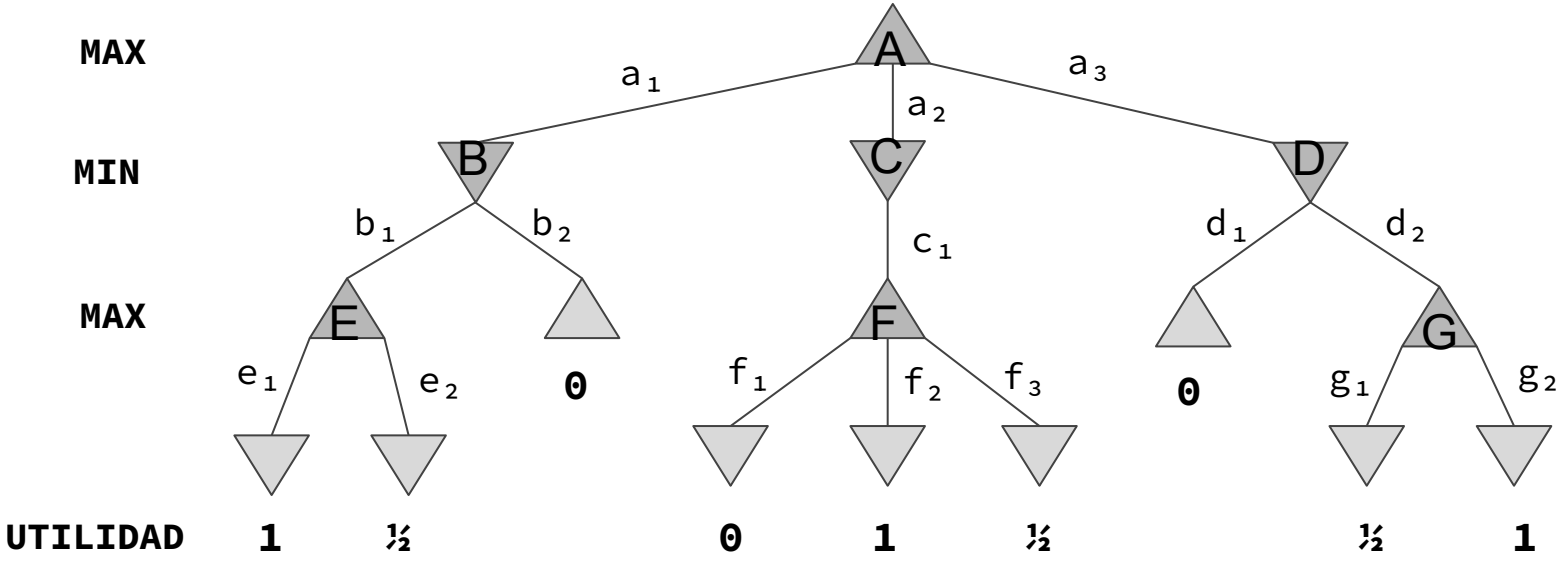
… y MIN juega **óptimamente**, jugará c_3 y MAX **pierde** (peor resultado que la estrategia minimax). Si MIN se puede **equivocar** y juega c_2, entonces MAX **gana** (mejor resultado que la estrategia minimax).



Valores minimax

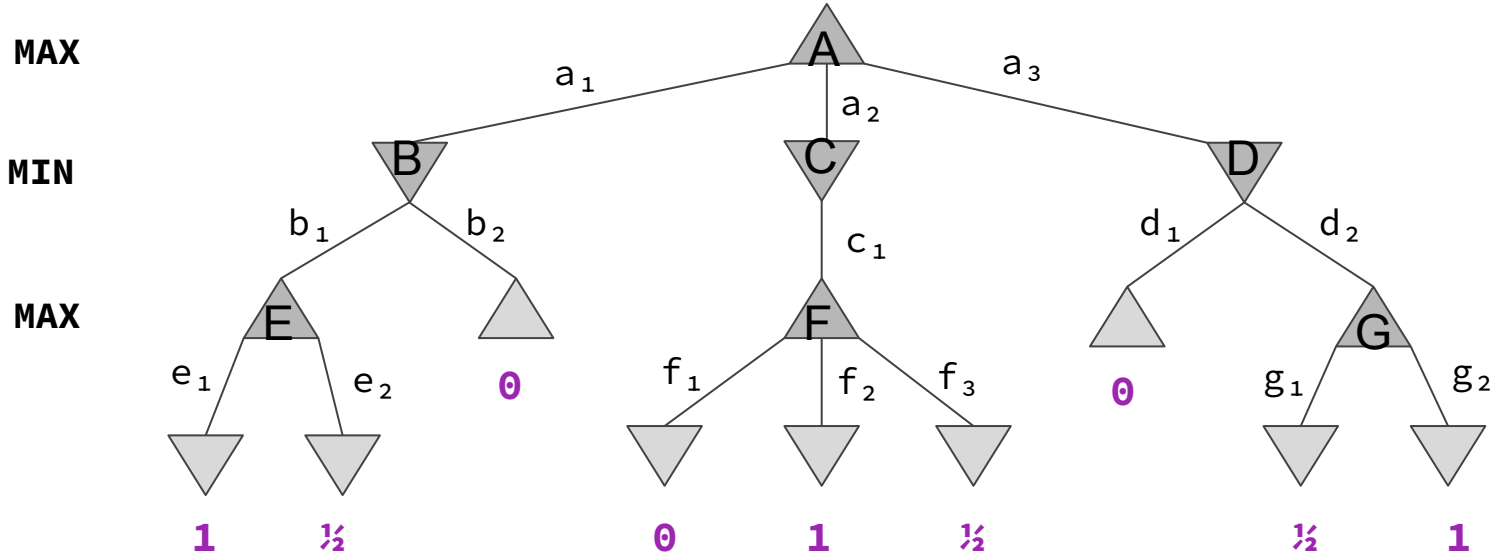
Ejercicio

¿Quién gana el siguiente juego? Suponer que MAX y MIN juegan óptimamente.



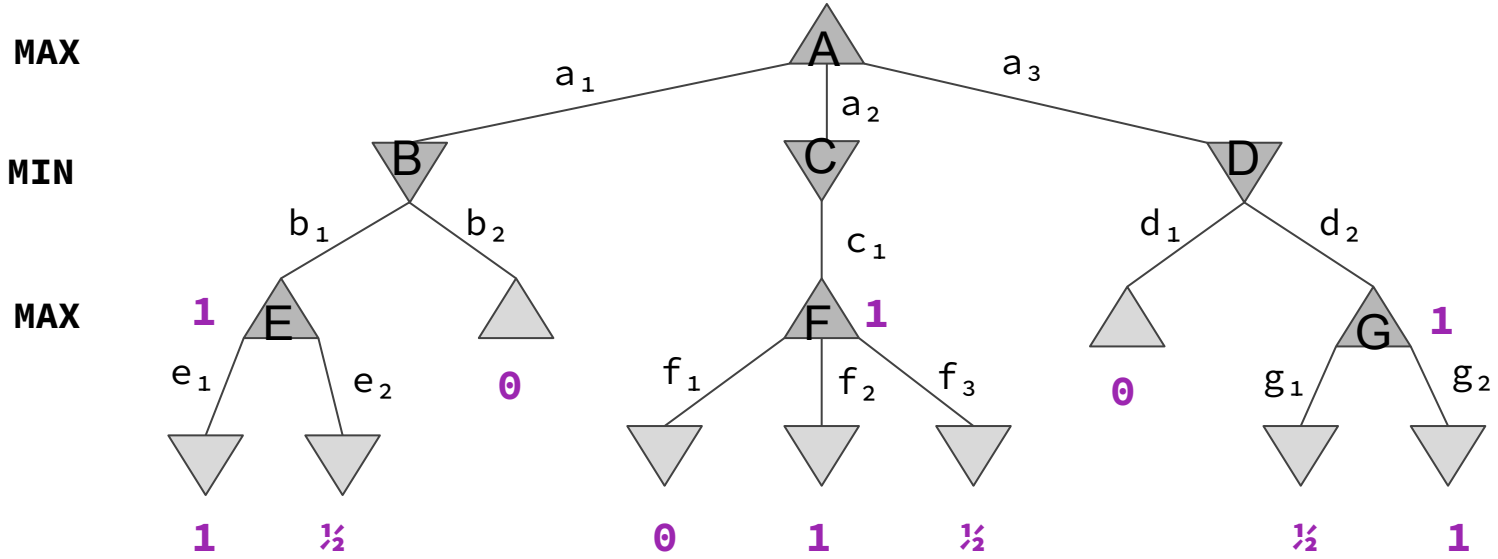
Respuesta

¿Quién gana el siguiente juego? Suponer que MAX y MIN juegan óptimamente.



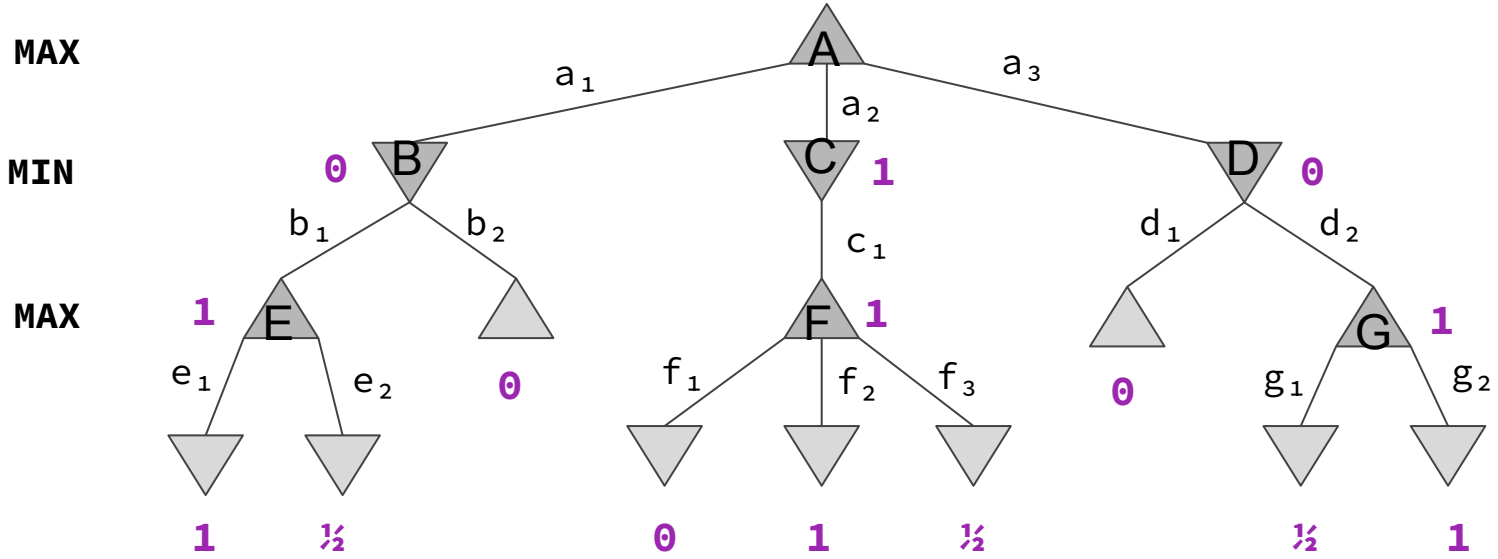
📝 Respuesta

¿Quién gana el siguiente juego? Suponer que MAX y MIN juegan óptimamente.



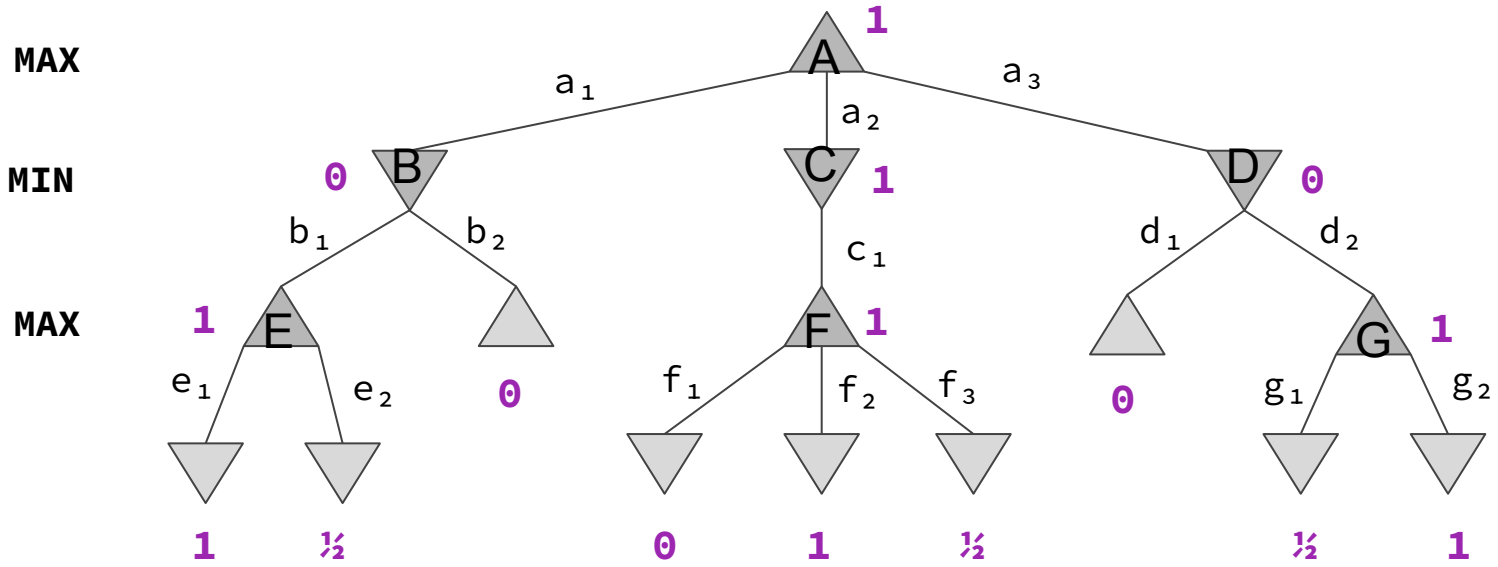
📝 Respuesta

¿Quién gana el siguiente juego? Suponer que MAX y MIN juegan óptimamente.



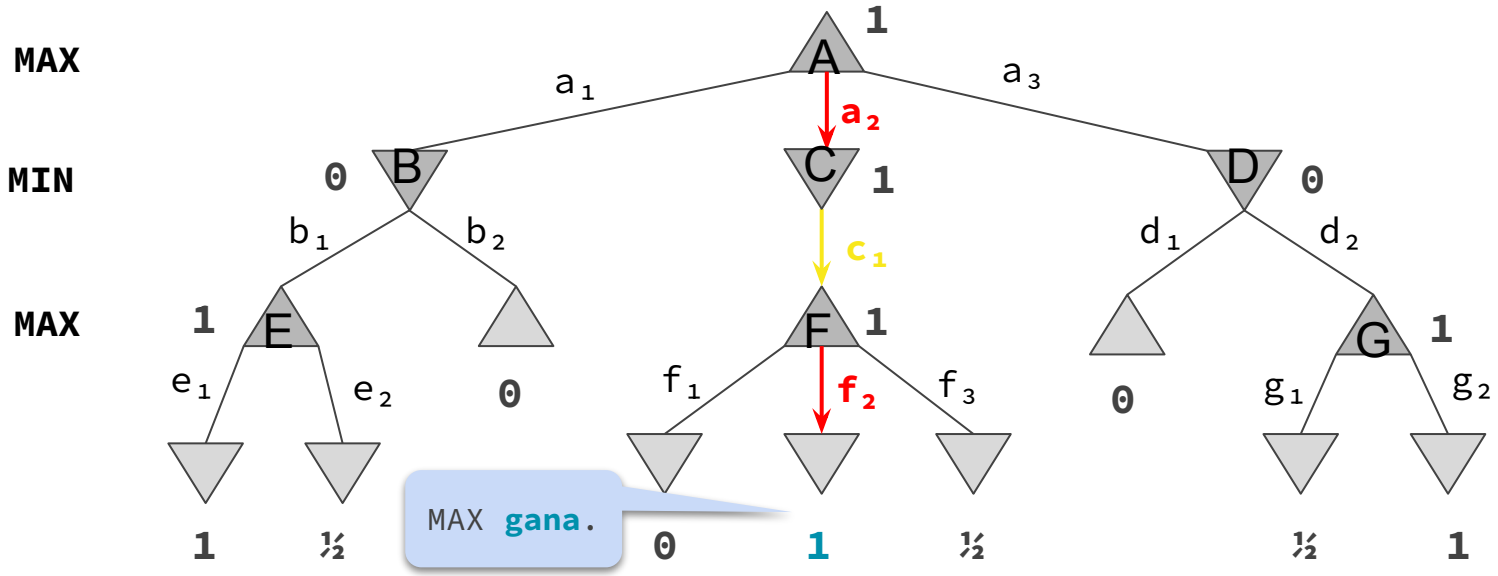
📝 Respuesta

¿Quién gana el siguiente juego? Suponer que MAX y MIN juegan óptimamente.



📝 Respuesta

¿Quién gana el siguiente juego? Suponer que MAX y MIN juegan óptimamente.



Implementación del algoritmo minimax

- ¿Cómo implementamos un algoritmo que calcule el valor minimax de todos los nodos del árbol de juego? 🤔
- Dado un nodo que no es hoja, para determinar su valor minimax es necesario conocer el de sus hijos.
- Es decir, hay que llamarse **recursivamente** en cada uno de sus hijos y elegir el máximo o mínimo valor, según si es el turno de MAX o MIN respectivamente.
- Veremos una implementación recursiva para este algoritmo.

💻 Algoritmo minimax

```
# Calcula recursivamente el valor minimax
# en un nodo MAX

function MINIMAX-MAX(problema, estado) return valor
    if problema.TEST-TERMINAL(estado) then
          return problema.UTILIDAD(estado,MAX)
    valor ← -∞
    forall acción in problema.ACCIONES(estado) do
          sucesor ← problema.RESULTADO(estado, acción)
          valor ← max(valor,
                      MINIMAX-MIN(problema, sucesor)
    return valor
```

```
# Calcula recursivamente el valor minimax
# en un nodo MIN

function MINIMAX-MIN(problema, estado) return valor
    if problema.TEST-TERMINAL(estado) then
          return problema.UTILIDAD(estado,MAX)
    valor ← +∞
    forall acción in problema.ACCIONES(estado) do
          sucesor ← problema.RESULTADO(estado, acción)
          valor ← min(valor,
                      MINIMAX-MAX(problema, sucesor)
    return valor
```

Algoritmo minimax

Las funciones son mutuamente recursivas.

La recursión desciende desde el estado actual hasta los estados terminales (donde la última llamada recursiva termina) y luego los valores minimax se propagan hacia arriba por el árbol a medida que la recursión retrocede.

```
# Calcula recursivamente el valor minimax
# en un nodo MAX

function MINIMAX-MAX(problema, estado) return valor
    if problema.TEST-TERMINAL(estado) then
        return problema.UTILIDAD(estado,MAX)
    valor ← -∞
    forall acción in problema.ACCIONES(estado) do
        sucesor ← problema.RESULTADO(estado, acción)
        valor ← max(valor,
                    MINIMAX-MIN(problema, sucesor))
    return valor
```

```
# Calcula recursivamente el valor minimax
# en un nodo MIN

function MINIMAX-MIN(problema, estado) return valor
    if problema.TEST-TERMINAL(estado) then
        return problema.UTILIDAD(estado,MAX)
    valor ← +∞
    forall acción in problema.ACCIONES(estado) do
        sucesor ← problema.RESULTADO(estado, acción)
        valor ← min(valor,
                    MINIMAX-MAX(problema, sucesor))
    return valor
```

💻 Algoritmo minimax

Algoritmo que implementa la estrategia minimax. Toma un estado y devuelve la acción a aplicar en ese estado.

```
function MINIMAX(problema, estado) return acción
   if problema.JUGADOR(estado) == MAX:
        sucs ← {acción: MINIMAX-MIN(problema, problema.RESULTADO(estado, acción))
                for acción in problema.ACCIONES(estado)}
        return max(sucs, key=sucs.get) # obtiene la clave con el mayor valor
   if problema.JUGADOR(estado) == MIN:
        sucs ← {acción: MINIMAX-MAX(problema, problema.RESULTADO(estado, acción))
                for acción in problema.ACCIONES(estado)}
        return min(sucs, key=sucs.get) # obtiene la clave con el menor valor
```

Performance de MINIMAX

- Hace una exploración **primero en profundidad** (recursiva) del árbol de juego.
- Sea un árbol de juego finito de altura ***m*** y factor de ramificación ***b***.
- **Consumo de tiempo**. Genera a lo sumo b^{m+1} nodos.
- **Consumo de memoria**. Almacena a lo sumo $b.m$ llamadas recursivas.

📖 Ejemplo

- En promedio, el ajedrez tiene un factor de ramificación de **35** y cada jugador hace **50** movimientos. Luego, el árbol de juego tiene aproximadamente

$$\mathbf{35^{100} \approx 10^{154}\ nodos}$$

  (aunque la cantidad de estados distintos es $\approx 10^{40}$).

- **Para juegos reales, el consumo de tiempo de MINIMAX es completamente impráctico.** Pero sirve de inspiración para algoritmos más sofisticados y para el análisis teórico.

Decisiones imperfectas en tiempo real

La profundidad en el árbol de juego a la que necesita llegar el algoritmo minimax a menudo no es práctica.

Típicamente las acciones deben decidirse en lapsos de tiempo acotados.

¿Cómo tomar la mejor decisión cuando no es posible computar una decisión óptima?

💡 Una idea revolucionaria...

- El algoritmo minimax tiene que descender hasta las hojas del árbol de juego.
- Una alternativa es cortar la búsqueda **anticipadamente** en un cierto nivel de profundidad y devolver la mejor decisión para ese árbol de juego parcial.
- ¿Cómo lo logramos? 🤔

Funciones evaluación

- Los estados no-terminales en un cierto nivel de profundidad se interpretan como terminales.
- A dichos estados, se le aplica una **función de evaluación** que retorna una **estimación del valor minimax**.
- El momento en el que se corta la búsqueda se determina con una función de **test de corte**.

Valor minimax estimado

Así, tenemos la siguiente aproximación para calcular el valor minimax para un estado *S* en una profundidad *d*:

```
H-MINIMAX(S,d) =
   { EVAL(S)                                                    SI TEST-CORTE(S,d)
   { max {H-MINIMAX(RESULTADO(S,A),d+1): A ∈ ACCIONES(S)}   SI JUGADOR(S) = MAX
   { min {H-MINIMAX(RESULTADO(S,A),d+1): A ∈ ACCIONES(S)}   SI JUGADOR(S) = MIN
```

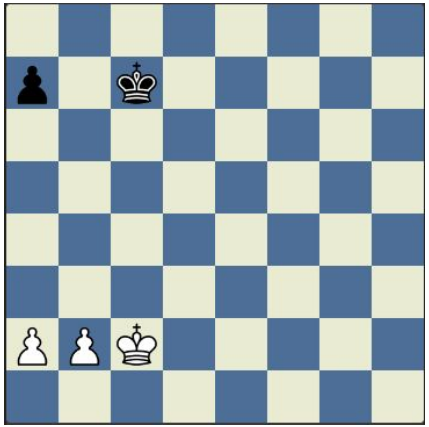
Valor minimax estimado

- ¿Cómo diseñar una buena función de evaluación (EVAL)?
  - Depende del juego (al igual que sucede con las heurísticas en las búsquedas informadas).
- Veremos dos técnicas generales.

Diseñando funciones de evaluación – Primera técnica

Se agrupan en una misma **categoría** (o clase de equivalencia) aquellos estados que tengan los mismos valores en ciertas **características**.

📖 **Ejemplo (Ajedrez).** La categoría **“2 peones vs. 1 peón”** contiene todos los estados con 2 peones (y el rey) blancos contra 1 peón (y el rey) negro.



Diseñando funciones de evaluación – Primera técnica

Luego, para cada categoría, se computa un **valor esperado** que refleje la proporción de sus estados que conducen a triunfos, empates y pérdidas.

📖 **Ejemplo (Ajedrez).** Nuestra experiencia sugiere que **72%** de los estados de la categoría **“2 peones vs. 1 peón”** llevan a una victoria (1 de utilidad), **20%** a una derrota (0) y **8%** a un empate (1/2), entonces una función de evaluación razonable para los estados de esta categoría devuelve el valor:

$$(0{,}72 \times 1) + (0{,}2 \times 0) + (0{,}08 \times ½) = 0{,}76$$

En la práctica, este análisis suele requerir muchas categorías y demasiada experiencia para estimar correctamente las probabilidades.

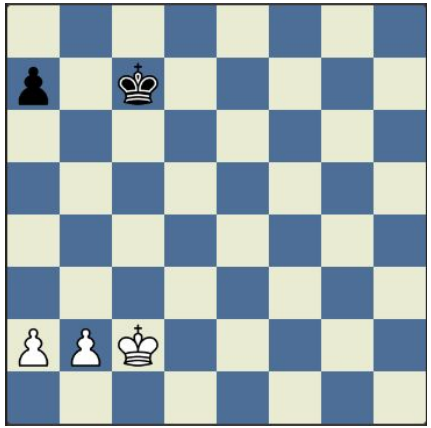
Diseñando funciones de evaluación – Segunda técnica

Se eligen ciertas **características** de los estados y se les asigna una contribución numérica, que luego se **combinan** para encontrar el valor total.

📖 **Ejemplo (Ajedrez)**. Las piezas tienen un **valor material**: cada peón vale 1, alfil y caballo valen 3, torre 5 y reina 9.

En este ejemplo, el valor total de los peones blancos es 2 y de los negros 1.

Otras características más complejas incluyen las posiciones de las piezas, el estado del juego (inicio o endgame), seguridad del rey, estructura de los peones, etc.



Diseñando funciones de evaluación – Segunda técnica

Las contribuciones se pueden sumar directamente para obtener la evaluación de un estado. Así, la función de evaluación es una **función ponderada lineal** (weighted linear function):

$$\textsc{Eval}(s) = w_1 f_1(s) + w_2 f_2(s) + \cdots + w_n f_n(s) = \sum_{i=1}^{n} w_i f_i(s) \,,$$

donde cada w_i es un peso y cada f_i es una característica.

Esta fórmula asume que la contribución de cada característica es **independiente** de las demás. Funciones más poderosas pueden incorporar términos no lineales.

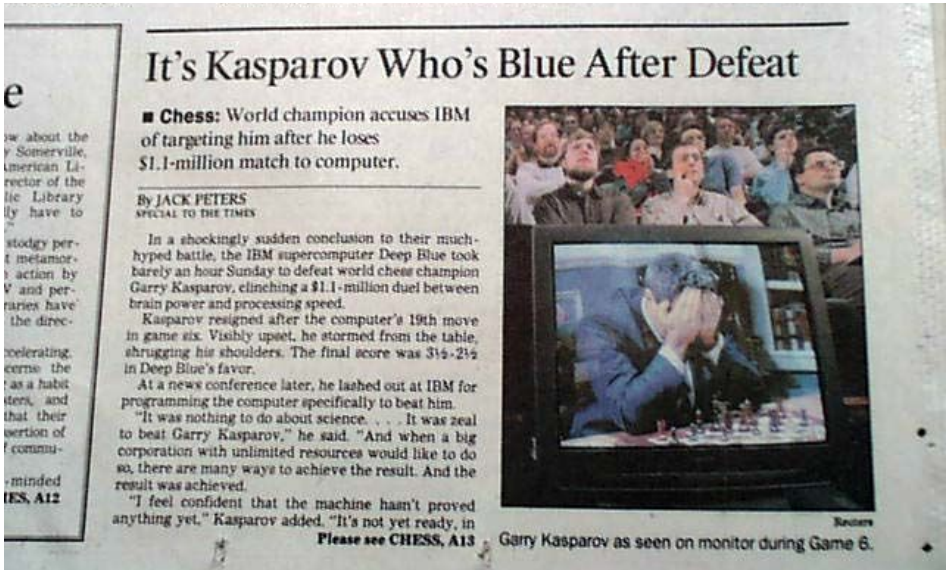
📝 Ejercicio

Dar el pseudocódigo del algoritmo H-MINIMAX, que incorpora una **función de evaluación** y un **test de corte** al algoritmo MINIMAX.

Performance de H-MINIMAX

- **Ya no garantiza encontrar una estrategia óptima.**
- Puede derrotar a oponentes subóptimos con una buena función de evaluación.
- El consumo de tiempo se puede reducir drásticamente con un test de corte apropiado.
- Mientras más capas (niveles) del árbol de juego se generen, más acertadas son las decisiones pero mayor el consumo de tiempo.

Deep Blue vs Garry Kasparov



It's Kasparov Who's Blue After Defeat

■ **Chess:** World champion accuses IBM of targeting him after he loses $1.1-million match to computer.

By JACK PETERS
SPECIAL TO THE TIMES

In a shockingly sudden conclusion to their much-hyped battle, the IBM supercomputer Deep Blue took barely an hour Sunday to defeat world chess champion Garry Kasparov, clinching a $1.1-million duel between brain power and processing speed.

Kasparov resigned after the computer's 19th move in game six. Visibly upset, he stormed from the table, shrugging his shoulders. The final score was 3½-2½ in Deep Blue's favor.

At a news conference later, he lashed out at IBM for programming the computer specifically to beat him.

"It was nothing to do about science. . . . It was zeal to beat Garry Kasparov," he said. "And when a big corporation with unlimited resources would like to do so, there are many ways to achieve the result. And the result was achieved.

"I feel confident that the machine hasn't proved anything yet," Kasparov added. "It's not yet ready, in

Please see CHESS, A13

Reuters

Garry Kasparov as seen on monitor during Game 6.

LOS ANGELES TIMES, 12 de mayo, 1997.

La supercomputadora Deep Blue de IBM vence en 6 partidas al gran maestro de ajedrez y campeón del mundo Garry Kasparov.

🔧 Tecnología de Deep Blue

- Búsqueda minimax con poda alfa-beta (próxima clase).
- Test de corte selectivo: no exploraba todas las ramas con la misma profundidad.
- Función de evaluación diseñada por expertos en ajedrez.
- Bases de datos de aperturas, ahorrando tiempo en las primeras jugadas.
- Bases de datos de finales precomputadas con la jugada óptima.
- Hardware especializado y paralelismo.

📚 Resumen

- Un **juego** se puede definir con un **estado inicial**, una función que determina de cuál **jugador** es el turno, las **acciones** permitidas en cada estado, el **resultado** de cada acción, el **test terminal** y la función de **utilidad** en estados terminales.
- Nos limitamos a juegos de **dos jugadores, de suma cero, con información perfecta**.
- El algoritmo **minimax** computa una **estrategia óptima** haciendo una exploración primero en profundidad del árbol de juego, asumiendo que ambos juegan óptimamente.
- Frecuentemente no es posible considerar el árbol entero, entonces se debe **cortar** la búsqueda en algún punto y aplicar una **función de evaluación** para estimar las utilidades.
- Programas computacionales han derrotado a campeones en ajedrez, damas y otros juegos. Todos estos programas incorporan en esencia las técnicas aquí presentadas. La efectividad depende fundamentalmente de cuántas capas del árbol es capaz de explorar en el tiempo límite.